🚀 模块深度剖析：PollingEngine.js

🧱 第一步：业务黑盒（产品说明书视角）

1. **核心职责**：`PollingEngine` 是整个工业物联网网关后端的**"心脏起搏器"。**它按照固定周期自动向每一台 GD32 环境控制器发起 Modbus TCP 通讯，将现场的温湿度、继电器状态等原始寄存器数据拉回来，经过解析后推送给前端大屏。同时为了保持 TCP 链路不被路由器/交换机空闲回收，它还负责定时发送**心跳包**（喂看门狗）。
2. **输入**：
   - **设备连接池（`DevicePool`）**：提供与每台 GD32 硬件之间已建立好的 Modbus TCP Socket 连接。
   - **轮询配置（`options`）**：如巡检间隔 `intervalMs`（默认 1000ms）、最大重试次数 `maxRetries`。
3. **输出**：
   - **JSON 数据流（`onData` 回调）**：解析后的 16 路温湿度、8 路 CO_2、4 路氨气、22 路继电器布尔状态——打包成 JSON 对象，经 WebSocket 广播到前端。
   - **控制报文**：将用户界面的"开关风机"、"触发 OTA 升级"等动作翻译成 Modbus 写寄存器指令，下发到硬件。
4. **使用场景举例**：
   - 养殖场管理员打开网页监控大屏，`PollingEngine` 正在后台每 30 秒自动采集一次所有温湿度传感器数据，前端仪表盘上的数值会"跳动刷新"。
   - 当管理员点击"开启 1 号风机"按钮时，`PollingEngine` 会立刻暂停常规轮询（`writeLock = true`），优先将继电器控制指令下发给 GD32 硬件，完成后再恢复采集。

🦴 第二步：静态骨架（代码结构图）

核心状态 / 变量

变量	类型	用途	C 语言类比
`isRunning`	Boolean	引擎总使能位	`TIM_CR1.CEN` 定时器使能
`isPolling`	Boolean	重入保护锁，防止上一轮还没跑完下一轮又进来	中断嵌套保护标志
`writeLock`	Boolean	互斥锁，控制指令执行期间暂停轮询	`__disable_irq()` 关全局中断
`timerId`	Number	setInterval 返回的定时器句柄	`TIM_HandleTypeDef`
`heartbeatCounter`	Number	心跳自增计数器 (0–65535)	软件看门狗喂狗值

deviceStatus	Map	记录每台设备的上次心跳和采集时间戳，实现分频	多任务分频计数器数组

核心方法

方法	作用	C 语言类比
start()	启动定时器、立刻执行首次轮询	HAL_TIM_Base_Start_IT()
stop()	清除定时器	HAL_TIM_Base_Stop_IT()
pollAllDevices()	for 循环遍历所有已启用设备	for (i=0; i<DEV_NUM; i++) 任务调度
pollDevice(key)	单台设备的"重连→心跳→采集"三级跳	状态机 switch-case
readBlock(key, blockName)	按数据块定义读取一段连续寄存器	HAL_UART_Receive()，按定义长度收帧
controlRelay(key, idx, val)	继电器控制（读→改→写→校验）	GPIO 位操作 + ReadModifyWrite
triggerOTA(key, version)	触发远程固件升级	IAP 引导区跳转指令
readConfig(key)	读取目标温湿度等配置参数	从 EEPROM 读取配置区

⚙️ 第三步：动态运转（生命周期与数据流）

1. 启动与循环机制

外部调用 `start()` 之后，引擎做了三件事：

1. 设置 `isRunning = true` （使能定时器）。
2. **立刻** 执行一次 `pollAllDevices()` （上电首次采集，不等待第一个周期到来）。
3. 挂载 `setInterval` ，以 `intervalMs` 为周期持续触发。

> 💡 ***C 类比**：这就像你在 `main()` 里先手动调用一次 `TIM_IRQHandler()` 确保首帧数据立刻可用，然后启动硬件定时器让后续自动触发。*

重入保护：每次定时器回调执行前，先检查 `this.isPolling` 和 `this.writeLock` 。如果上一轮还没跑完（网络超时卡住），或者正在执行控制指令，就直接 `return` ，不进入业务逻辑。

> 💡 ***C 类比**：`setInterval` = `vTaskDelay(t) + while(1)` （带固定延时的无限循环任务）。`isPolling` 就像 FreeRTOS 中的**二值信号量**——进入中断前 `Take` ，退出时 `Give` 。*

2. 分频并发策略

引擎内部通过 `deviceStatus` Map 存储每台设备的 `lastHeartbeat` 和 `lastSensorPoll` 时间戳：

- **心跳**：`now - lastHeartbeat >= 5000` → 每 5 秒写一次寄存器 0x0000。
- **数据采集**：`now - lastSensorPoll >= 30000` → 每 30 秒全量读取环境传感器 + 硬件状态。

> 💡 ***C 类比**：这是经典的**多速率任务调度**。在裸机开发中，你可能用一个 1ms 的 SysTick 做基础节拍，然后用 `if (tick % 5000 == 0)` 来分出 5 秒和 30 秒任务——本质完全相同。*

3. 异常与边界处理

场景	代码防线	C 类比
设备断线	`pollDevice()` 入口处检查 status，非 CONNECTED 则调 `pool.connect()` 重连	TCP 断线重连状态机
重连后需等待	`setTimeout(resolve, 300)` 等 300ms 让物理层稳定	延时 debounce
上一轮卡死	`isPolling` 锁防重入	中断嵌套保护
心跳/读取失败	提前更新时间戳，即使本次报错也会等下一周期才重试	WDT 错误计数器只增不减
内存泄漏风险	`stop()` 调用 `clearInterval` 销毁定时器	`HAL_TIM_Base_Stop_IT()`

🎯 第四步：执行主线

① Happy Path（顺风局：一次完整的"心跳 + 采集"周期）

```
// [执行主线] 模拟 pollDevice(key) 的一次完整周期
async pollDevice(key) {
    // 步骤1: 检查连接——断线自动重连
    const status = this.pool.getStatus(key);
    if (status.status !== 'CONNECTED') {
        const ok = await this.pool.connect(key);      // 尝试 TCP 重连
        if (!ok) return;                              // 连不上就跳过
        await new Promise(r => setTimeout(r, 300));   // 300ms 冷却，等链路稳定
    }

    const now = Date.now();

    // 步骤2: 心跳——喂看门狗 (每 5s)
    if (now - dStatus.lastHeartbeat >= 5000) {
        dStatus.lastHeartbeat = now;                  // 【关键】先更新时间戳再写，防止反复
重试
        this.heartbeatCounter = (this.heartbeatCounter + 1) % 65536;
        await this.pool.writeRegister(key, 0x0000, this.heartbeatCounter);
    }
```

```
    // 步骤3: 数据采集——拉传感器 (每 30s)
    if (now - dStatus.lastSensorPoll >= 30000) {
        dStatus.lastSensorPoll = now;                  // 同理先更新
        const envData = await this.readBlock(key, 'BLOCK_ENV');   // 0x1001-0x1048
        const hwData  = await this.readBlock(key, 'BLOCK_HW');    // 0x4001-0x400B
        const parsed  = this.parser.parseSensorData({ env: envData, hw: hwData });
        this.onData(key, parsed);                      // 步骤4: 推送给 WebSocket 层
    }
}
```

② Edge Cases（逆风局：防御性编程）

- **重入拦截**（第 62-76 行）：

  ```
  if (this.isPolling) {
      console.log('Overlap detected, skipping this tick.');
      return;  // 上一轮还没跑完，直接跳过
  }
  this.isPolling = true;
  try { await this.pollAllDevices(); }
  finally { this.isPolling = false; }  // 无论成功失败都释放锁
  ```

  为什么这样写：网络超时可能让 `await` 挂住 3-5 秒，远超 1 秒的定时器周期。没有这个保护，就会出现 3 个轮询同时在排队向同一台设备发 Modbus 报文，直接把协议栈打爆。

- **互斥控制**（第 223-259 行）（`controlRelay` 的"读→改→写→验"四步原子操作）：

  ```
  this.writeLock = true;     // 步骤1: 关全局中断
  try {
      const resp = await read(0x5001, 2);   // 步骤2: 读当前
      let val = parser.generateRelayControl(val, idx, on); // 步骤3: 改位
      await write(0x5001, [high, low]);      // 步骤4: 写回
      // 步骤5: 验证读回（如不一致则重写一次）
  } finally {
      this.writeLock = false; // 步骤6: 恢复全局中断
  }
  ```

  为什么这样写：如果写入期间定时器又触发了一次心跳写入，两个写操作同时打到 Modbus 总线上，GD32 的 TCP 协议栈会混乱甚至断连。

③ 关键语法点拨（跨界降维打击）

> 💡 *`setInterval(async () => {...}, ms)` 在 JS 里，它就像嵌入式的 `vTaskDelay(t) + while(1)` ——一个带固定延时的无限循环任务。但要注意：JS 的 setInterval **不关心** 回调是否执行完毕就会触发下一次，这就是为什么需要 `isPolling` 锁来做重入保护。*

> 💡 `async/await` 相当于 C 中的 `xSemaphoreTake(sem, portMAX_DELAY)`。代码执行到 `await` 时会原地挂起（让出 CPU），直到网络操作返回结果才继续往下走。期间不会阻塞其他事件循环中的任务。

> 💡 `try { ... } finally { this.isPolling = false; }` 就像 C 里的 `__enable_irq()` ——无论前面的业务代码跑飞还是抛异常，`finally` 块**百分之百**会执行，保证锁一定会被释放，绝不会死锁。这等价于嵌入式里的"无论如何都要恢复全局中断"。

> 💡 `Map` **数据结构** 对标 C 里用设备 ID 做下标访问结构体数组——`deviceStatus[deviceId].lastHeartbeat`。相比普通对象，`Map` 的查找性能是 O(1)，并且 key 可以是任意类型（不像 C 的 struct 数组下标只能是整数）。

> 💡 `(this.heartbeatCounter + 1) % 65536` 这就是一个 **16 位无符号自增计数器**，与 STM32 的 TIM_CNT 寄存器行为完全一致——到 0xFFFF 后自动回绕到 0。